第一周完整自学讲义

Python 工程化 × Linux 系统基础

四维学习路线·第 01 周·学习之旅

版本：2026-10-29 | 建用：27 小 | 主目：Linux 系统信息采集器

这不是知识点清单，而是一份从概念、例子、实验到自测的教材。所有必修概念都在正文解释；官方资料只用于核对和延伸，不要求你另外看完整课程。参考项目源码、自动化测试、分步练习和记录模板随资料包提供。

本周最终能力： 不看完整答案，能够解释并实现“读取系统状态 → 校验和组织数据 → 保存 JSON → 记录异常 → 自动化测试”的小工具。会运行参考代码是起点，不是结课标准。

本周边界： 不安装大模型、不训练神经网络、不购买开发板。四个方向不是第一周各学一套框架，而是先共用同一套运行、观察、测试和排错方法。

00 | 先看这里：顺序、重点和每日产出

00.1 四个方向怎样落到第一周

方向	本周做什么	暂时不做什么
计算机基础与质量工程	Python、Linux、文件、进程、日志、测试和 Git	数据库、完整网络协议、复杂算法专项
AI 应用与模型	分清模型、程序与系统；保存可追溯数据；区分事实和推测	RAG、Agent、微调和模型训练
AI Infra 与部署	查看 CPU、内存、磁盘、进程；理解采样与退出状态	Docker、Kubernetes、GPU 集群
软硬件协同	看懂架构和设备信息；理解操作系统可见范围	驱动开发、电路设计、硬件采购

这一周的监控报告，将来可以扩充成模型推理监控、设备运行记录和故障诊断证据。注意：采集器不是 AI 模型，规则告警也不是学习算法；这是为后续系统建立工程基础。

00.2 先读哪一部分

第一次阅读严格按 00 → 01 → 02 → 03 → 04 → 05 → 06 → 07 → 08 → 09 → 10 → 11 → 12。 先学概念和小实验，再读综合项目源码。附录 A 是答案，先作答再看；附录 B 是速查；附录 C 是资料来源。

每个知识点按三个层次掌握：**必会** 表示能独立使用并解释；**理解** 表示看懂例子、能改小功能；**选学** 本周可暂不实现。生成器、装饰器、SSH 和服务管理不要求第一周达到生产运维熟练度。

顺序	章节	一句话摘要	学完的证据
1	01 环境与系统地图	分清硬件、内核、终端、Shell、Python 和虚拟环境	能确认自己用的是哪个 Python
2	02 Linux 文件与命令	知道文件在哪，命令如何组合，错误怎样留下证据	完成目录、管道、权限实验
3	03 Python 数据与函数	把需求拆成明确输入输出的函数	独立写出阈值判断与校验
4	04 文件与 JSON	把内存中的数据保存为可交换文件	写入再读出，内容保持一致
5	05 模块、包与类	组织代码，而不是把所有逻辑堆在一个脚本	能说明入口、采集、存储各自职责
6	06 异常与日志	区分能继续的局部失败和必须退出的整体失败	留下一条有上下文的错误记录
7	07 进程、资源与硬件	正确解释采样数据，不凭一次读数下结论	用 Linux 与 Python 交叉观察
8	08 三种 Python 机制	理解按需生成、功能包装与资源自动清理	能读懂并修改三个小例子
9	09 自动化测试	正常、边界、异常都要验证	自己设计至少 5 个测试
10	10 Git 与实验文档	保存可解释的改动和可复现实验	至少 3 次有意义的提交
11	11 综合项目	分阶段搭建采集器并做故障实验	JSON、日志、测试、报告齐全
12	12 结课自测	用独立操作检验掌握程度	完成实操和闭卷解释

00.3 六天学习，留一天复盘

学习日	内容	参考时间分配	当日产出
Day 1	00、01、02；10.1 的 Git 初始化	理解 1.5h，操作 2h，复盘 0.5h	环境记录、命令实验
Day 2	03、04；11.2 的单文件版本	理解 1.5h，编写 2h，复盘 0.5h	首份 JSON 报告
Day 3	05、06	理解 1.5h，改写 2h，复盘 0.5h	模块、异常与日志
Day 4	07；SSH/服务管理了解即可	理解 1.5h，测量 2h，复盘 0.5h	资源解释与进程记录

学习日	内容	参考时间分配	当日产出
Day 5	08、09、10	理解 1.5h，测试 2h，复盘 0.5h	自动化测试、版本记录
Day 6	11、12；附录 A 对答案	项目 3h，故障 2h，验收 2h	完整基础版及实验报告
Day 7	查漏补缺或休息	不安排强制新知识	补做未通过项目

27 小时是课程安排，不是对你个人学习速度的承诺。遇到环境问题可以放慢；不要为赶进度跳过“独立写、故意弄错、解释结果”这三步。

00.4 资料包怎样使用

Week1_Handbook.pdf 适合连续阅读和打印；Week1_Handbook.html 有章节导航，适合查找和复制代码；Week1_Handbook.docx 可自行批注。lab/ 是参考工程，exercises/ 是分步任务和答案，templates/ 保存环境、故障与周报模板。

正文中的 Bash 代码在 **Ubuntu/Linux 终端** 运行；标为 PowerShell 的代码只在 Windows PowerShell 运行；Python 代码保存成 .py 文件后由 Python 执行。不要把三种窗口混用。代码块中的输出说明不是要输入的命令。

安全范围： 只操作你自己的实验目录和自己启动的进程。不在学校共享服务器做压力测试；不修改系统权限、驱动或内核参数；不上传密码、密钥、用户名、主机地址或含个人信息的日志。

01 | 环境与系统地图：程序到底在哪里运行

本章摘要： 先搞清“我在哪个系统、哪个目录、哪个 Python 环境”，再开始写程序。环境可解释，后面的错误才容易定位。

01.1 硬件、内核、应用分别做什么【必会】

CPU 执行通用计算指令；内存保存程序正在使用的数据；磁盘等持久化介质保存文件。操作系统内核负责进程调度、内存管理、文件系统与设备访问。应用程序通过运行时、库以及系统调用请求这些能力。

例如 Path('report.json').write_text(...) 看起来只有一行 Python，背后仍要经过文件系统与操作系统的写入路径。文件是否允许写、目标目录是否存在、存储空间是否足够，都可能影响结果。因此“Python 语法没错”不等于“程序一定成功”。

Linux 严格说是内核；Ubuntu 是把内核、命令行工具、包管理器等组合起来的发行版。本周使用 Ubuntu 风格命令，是为了让实验路径一致，不是说所有 Linux 系统命令都完全相同。

01.2 终端、Shell、Python 不要混淆【必会】

终端是输入输出的交互窗口；Shell 是解释命令的程序，例如 Bash；Python 解释器负责执行 Python 代码。你在终端输入 python first.py，首先是 Shell 找到 Python，再由 Python 读取脚本。

看到 >>> 通常说明进入了 Python 交互环境。此时输入 cd 或 ls 会报错，因为它们不是 Python 语法。输入 exit() 返回 Shell。Shell 提示符中的 $ 或 # 不需要复制；本讲义代码块已省略提示符。

01.3 选择实验环境【必会】

已有 Linux：直接使用个人账户和个人目录。Windows：优先使用 WSL2 中的 Ubuntu；不是在 PowerShell 里执行 Linux 教程。macOS：Python 部分可做，但 /proc、free 等实验需要 Linux 虚拟机或已授权的 Linux 环境；本讲义不把 macOS 输出当作 Linux 输出。

尚未安装 WSL 的 Windows 用户，在管理员 PowerShell 中执行：

```
wsl --install -d Ubuntu
```

按提示重启和创建 Linux 用户。已有 WSL 时不必重新安装，可在 PowerShell 用 wsl -l -v 查看发行版与版本。安装可能受 Windows 版本、虚拟化、管理员权限和网络影响，以微软安装说明为准。[S01]

WSL2、容器、虚拟机观察到的资源可能只是所在环境可见的部分，不能自动等同于整台物理主机。设备枚举不完整时，先写明环境限制，而不是判断电脑没有那个硬件。

01.4 建立自己的工作目录【必会】

下面所有 Linux 命令都在 Ubuntu 终端运行。先安装本周必要工具；sudo 只用于这一步的系统包安装。

```
sudo apt update
sudo apt install python3 python3-venv python3-pip git
mkdir -p ~/ai-systems-lab/week1
cd ~/ai-systems-lab/week1
pwd
python3 --version
git --version
```

~ 表示当前用户主目录。mkdir -p 可创建缺失的父目录；cd 改变当前工作目录；pwd 显示当前目录。之后创建相对路径文件时，默认相对于这个工作目录，而不是相对于你脑中“项目应该在的位置”。

本教材代码使用 Python 3.10+ 语法；优先沿用现有受支持的 Python，不必为了课程替换系统 Python。参考工程记录了实际测试版本，见资料包 VALIDATION.txt，不是声称所有版本都已实测。

01.5 虚拟环境隔离什么【必会】

虚拟环境让项目使用一套独立的 Python 包安装位置。它不是虚拟机，不提供独立 Linux 内核，也不会自动切换成任意 Python 版本。创建环境所使用的解释器决定了基础版本。[S02]

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install psutil==7.2.2 pytest==9.0.2
python -c "import sys; print(sys.executable)"
python -m pip --version
```

逐句理解：-m venv 让该 Python 执行标准库的环境创建模块；.venv 是目录名；source 在当前 Shell 中执行激活脚本；激活主要调整 PATH 等环境信息；python -m pip 明确把安装动作交给当前 Python 的 pip，减少“装到 A、运行在 B”的混乱。

sys.executable 应指向项目 .venv 中的解释器。新开终端后通常需要重新激活。退出使用 deactivate。不要 sudo pip install，不要为解决依赖错误删除系统 Python。

以上版本用于本资料的复现，不代表“最新版本推荐”。安装失败时先看 Python 版本、网络、错误原文，不能盲目反复升级全部依赖。资料包的 lab/requirements.txt 给出同一组依赖。

01.6 认识三类依赖与复现记录【必会】

json、pathlib、logging 属于 Python 标准库，通常不需要额外 pip 安装。psutil 和 pytest 是第三方包。monitor 是本项目自己编写的包。混淆这三类，会导致无意义的安装或导入错误。

```
python -m pip list
python -m pip freeze > requirements-local.txt
uname -a
uname -m
```

pip freeze 记录当前环境的已安装包版本，便于复现，但不是对操作系统、编译器、硬件或所有平台兼容性的完整锁定。把操作系统、Python、依赖版本一起写进环境记录。

01.7 怎样创建并运行一个 Python 文件【必会】

使用已有文本编辑器新建 .py 文件即可，保存时不要变成 .py.txt。没有编辑器时，可以在 Linux 终端复制下面整段；Shell 会把两个 PY 标记之间的内容写进指定文件。

```
mkdir -p scratch
cat > scratch/hello.py <<'PY'
print("Hello, week 1")
PY
python scratch/hello.py
```

预期输出 Hello, week 1。末尾单独一行的 PY 是输入结束标记，不是 Python 代码。此操作会覆盖你自己实验目录中的同名 hello.py，已有重要内容时先另取文件名。

本章练习 E01： 关闭终端后重新进入项目并激活环境，打印解释器路径。解释“为什么执行 pip install 成功后仍可能 import 失败”。通过标准：你能先核对解释器，而不是立刻重新安装整个系统。

02 | Linux 文件、Shell、权限与命令组合

本章摘要： 命令不是一串要背的缩写，而是“查位置、看内容、筛证据、控制输出、检查权限”的工具。

02.1 目录地图与绝对/相对路径【必会】

绝对路径从 / 开始，例如 /home/student/report.json；相对路径从当前目录解释，例如 reports/report.json。. 是当前目录，.. 是父目录。Linux 文件名区分大小写；Report.json 与 report.json 可以是不同文件。

路径	用途	本周怎样使用
/	整个目录树的根	与用户主目录 ~ 区分
/home	普通用户主目录常见位置	放个人学习项目
/etc	系统配置的常见位置	只了解，不随意修改
/var/log	系统/服务日志常见位置	能读则查看，权限不足就记录
/proc	内核提供的运行信息接口	读取内存与进程信息
/dev	设备节点常见位置	理解应用访问设备的入口
/tmp	临时目录	不把重要成果只放这里

/proc 中许多内容是内核按读取时的状态生成的，不是普通磁盘文件；它既有信息接口，也有配置接口。本周只读，不向 /proc/sys 写入。[S03]

02.2 最小文件操作实验【必会】

```
cd ~/ai-systems-lab/week1
mkdir -p scratch reports logs
printf 'alpha\nbeta\n' > scratch/demo.txt
ls -lah scratch
cat scratch/demo.txt
cp scratch/demo.txt scratch/copy.txt
mv scratch/copy.txt scratch/renamed.txt
```

```
head -n 1 scratch/demo.txt
wc -l scratch/demo.txt
```

ls -lah 同时显示详细信息、隐藏文件和便于阅读的大小。以点开头的文件通常在默认 ls 中隐藏，.venv 与 .gitignore 都属于此类。cp 复制，mv 改名或移动。覆盖同名文件前确认目标。

cat 适合看短文件；head 看开头；tail 看末尾；less 适合翻阅长文件，按 q 退出。wc -l 计换行符，因此不能把所有文件的行数都简单等同于人眼看到的段落数。

除只自己建的文件：rm scratch/renamed.txt。不要练习 sudo rm -rf，也不要把未知路径直接复制到删除命令中。

02.3 标准输入、输出、错误与管道【必会】

一个命令常有三条通道：标准输入 stdin（0），正常输出 stdout（1），错误输出 stderr（2）。> 默认重定向 stdout；2> 重定向 stderr；>> 追加而不是覆盖。| 默认把左边 stdout 接到右边 stdin，不会自动接收 stderr。[S04]

```
printf 'INFO start\nERROR disk\nINFO done\n' > scratch/app.log
grep 'ERROR' scratch/app.log
cat scratch/app.log | grep 'INFO' | wc -l
ls /week1_nonexistent_path 2> scratch/error.txt
cat scratch/error.txt
```

第二个管道的预期结果是 2：先输出文件，再筛选 INFO，再计数。实际工作中也可直接 grep 'INFO' scratch/app.log，不必为了练管道给每个命令都加 cat。

command > all.log 2>&1 表示先把 stdout 指向文件，再让 stderr 指向当前 stdout；顺序有意义。> 会先打开并截断文件，所以不要写 cat a.txt > a.txt，这会破坏原内容。[S04]

02.4 退出状态与“命令成功”的含义【必会】

Shell 用退出状态判断命令结果。通常 0 表示成功，非 0 表示失败或特定状态。echo $? 读取上一条命令的退出状态，必须紧跟目标命令；中间运行了别的命令，读到的就变了。

```
ls /week1_nonexistent_path
echo $?
python -c "raise SystemExit(3)"
echo $?
```

最后预期看到 3。程序只打印“失败”却返回 0，会误导自动化脚本。a && b 表示 a 成功才执行 b；a || b 表示 a 失败才执行 b。管道默认状态通常取最后一个命令，本周先避免用复杂管道包装重要的成功/失败判断。

02.5 环境变量、PATH 与引号【必会】

环境变量是传给进程的字符串配置。PATH 是 Shell 查找外部可执行文件的目录列表。普通 Shell 变量不一定传给子进程，export 使其进入后续子进程环境。[S05]

```
export LAB_NAME=week1
printf '%s\n' "$LAB_NAME"
printf '%s\n' '$LAB_NAME'
command -v python
python -c "import os; print(os.getenv('LAB_NAME'))"
```

双引号中的变量会展开，单引号保留字面量；上面两个 printf 的结果不同。路径含空格时加双引号，例如 cd "my notes"。环境变量不是密码保险箱，不能把真实 API 密公截或 Git 历史。

02.6 权限：文件和目录不是同一套直觉【必会】

ls -l 中的 -rw-r--r--：第一个字符表示类型，随后每三位分别属于所有者、组、其他人。r/w/x 是读、写、执行。数值权限中 r=4、w=2、x=1，644 表示所有者 6=读写，其余 4=只读。

对目录，r 主要是列出条目，x 是穿过/访问目录中的路径，w 关联创建或删除条目。删除文件通常取决于父目录权限，并不只看文件本身是否可写。本周理解这一点即可，不扩展 ACL 与安全模块。[S06]

```
printf 'print("permission demo")\n' > scratch/permission_demo.py
chmod 600 scratch/permission_demo.py
ls -l scratch/permission_demo.py
python scratch/permission_demo.py
```

这个文件没有 x 权限，仍可由 Python 读取并执行；python file.py 与 ./file.py 不是同一条路径。后者需要合适的执行权限和解释器声明。不要用 chmod 777 作为所有权限问题的解决方案。

02.7 SSH 与服务管理【理解，不作为必做环境】

SSH 用于通过加密连接登录已授权远程机器。基本形式是 ssh user@host，非默认端口用 -p 指定。你需要真实的授权地址与账号；本周不需要租服务器。初次连接应核实主机指纹，不应关闭主机校验来省事。[S07]

后台进程不自动等于受管理的服务。systemd 环境中，systemctl status 服务名 查看服务状态，journalctl -u 服务名 查看该单元日志；服务的启动、重启通常还涉及权限。没有这个服务或系统没有以 systemd 运行，会得到不同结果。WSL 的 systemd 支持取决于配置与环境，本周无需为此改系统。[S08]

本章练习 E02： 建立 scratch/notes.txt，写入三行，筛出其中包含 ERROR 的行并保存；再让一次故意失败的命令把错误写入另一文件。解释 >、>>、2> 的区别。答案在附录 A。

03 | Python 数据与函数：把需求变成明确的输入输出

本章摘要： 先会表示数据、选择分支、重复处理，再把逻辑封装为函数。目标不是背完 Python 语法，而是能写出可解释、可测试的小功能。

本章短例子可分别保存到 scratch/ 下的 .py 文件，然后在项目目录运行 python scratch/文件名.py。同一个例子的定义与调用必须放在一起，不能只复制最后一行。

03.1 变量、对象与基本类型【必会】

Python 变量是绑定到对象的名字。cpu = 25.0 表示名字 cpu 指向一个浮点数对象，而不是创建一个永远只能装浮点数的盒子。对象有类型，变量名之后可以重新绑定，但随意改变同名变量的含义会让代码难读。

类型	示例	本周用途
int	8	逻辑 CPU 数、进程号
float	25.5	使用率、采样间隔
str	"Linux"	系统名称、路径、错误说明
bool	True / False	条件是否满足
None	None	没有结果或暂时无法获取
list	[10, 20, 30]	一组读数或进程记录
dict	{"cpu": 25.5}	带字段名的结构化报告

【代码目录准备（必须先做）】在 Ubuntu/WSL 终端输入以下命令，先创建文件夹并进入练习目录。

```
mkdir -p ~/ai-systems-lab/week1-practice/chapter_code
cd ~/ai-systems-lab/week1-practice
ls -ld chapter_code
```

只有父目录存在才能保存 chapter_code/ 下的 .py 文件；执行 nano 时，路径相对于 pwd 示的前目。料包中的 chapter_code 已自带源码。

03.1 完整可运行实验（新增）：先在项目目录执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/03_01_basic_types.py，粘贴下面整段代码，按 Ctrl+O、回车、Ctrl+X 保存，再执行 python chapter_code/03_01_basic_types.py。

```
cpu_count = 8
cpu_percent = 25.5
system_name = "Linux"
is_healthy = True
last_error = None
samples = [10.0, 20.0, 30.0]
report = {"cpu_percent": cpu_percent, "system": system_name}

for name, value in [("cpu_count", cpu_count), ("cpu_percent", cpu_percent),
```

```
            ("system_name", system_name), ("is_healthy", is_healthy),
            ("last_error", last_error), ("samples", samples), ("report", report)]:
    print(name, "=", value, "; 类型:", type(value).__name__)

print("是否没有错误信息:", last_error is None)
print("报告里的 CPU 使用率:", report["cpu_percent"])
```

预期能看到 cpu_count 类型为 int、cpu_percent 为 float、last_error 为 NoneType、samples 为 list、report 为 dict。这些是课堂模拟数据，不是对电脑的实时采集。

None 不等于 0。CPU 为 0% 表示这个窗口内测得接近空闲；CPU 为 None 表示没有可用结果。把采集失败写成 0，会制造“系统很健康”的假象。

字符串 "80" 不是数值 80。来自环境变量和文件的数据经常是字符串，需要显式转换，例如 float("80")。转换也会失败，输入 "eighty" 会抛出 ValueError，应在输入边界处理。[S09]

03.2 容器与可变性【必会】

列表适合有顺序的一组数据，字典适合按字段名查值。列表和字典是可变对象，两个名字可能指向同一份数据。

```
samples = [10.0, 20.0]
alias = samples
alias.append(30.0)
print(samples)          # [10.0, 20.0, 30.0]

copy = samples.copy()
copy.append(40.0)
print(len(samples))     # 3
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_01.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_01.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_01.py
```

03.2 完整对照实验（新增）：在项目目录执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/03_02_mutability.py，粘贴代码并保存，然后执行 python chapter_code/03_02_mutability.py。

```
samples = [10.0, 20.0]
alias = samples
alias.append(30.0)
print("原列表:", samples)
print("别名:", alias)
```

```
print("是否同一对象:", alias is samples)

copied = samples.copy()
copied.append(40.0)
print("制后原列表:", samples)
print("复制列表:", copied)
print("是否同一对象:", copied is samples)

nested = [[1, 2], [3, 4]]
shallow = nested.copy()
shallow[0].append(5)
print("拷后原嵌套列表:", nested)
```

预期：alias is samples 为 True，copied is samples 为 False；原列表不含 40.0，但嵌套列表的内部元素会受浅拷贝修改影响。

alias = samples 没有复制列表。.copy() 是浅复制；嵌套列表或字典中的内部对象仍可能共享，本周先不要把浅复制误认为深复制。调试时“我只改了另一个变量”并不保证原数据没变。

```
metrics = {"cpu_percent": 25.5, "memory_percent": 60.0}
print(metrics["cpu_percent"])
print(metrics.get("gpu_percent"))   # None
for name, value in metrics.items():
    print(name, value)
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_02.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_02.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_02.py
```

dict[key] 在缺少字段时抛出 KeyError；.get(key) 默认返回 None。必需字段用明确校验，不要为了不报错，把每个字段都 .get(..., 0)。这会掩盖数据格式错误。[S10]

03.3 分支、循环与布尔条件【必会】

if 决定做哪条路径，for 逐个遍历对象，while 在条件满足时继续执行。Python 用缩进表示代码块，建议每级 4 个空格，不混用 Tab。

```
samples = [35.0, 80.0, 92.5]
for percent in samples:
    if percent >= 80.0:
        state = "high"
    else:
```

```
    state = "normal"
  print(percent, state)
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_03.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_03.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

python chapter_code/code_03.py

预期依次输出 normal、high、high。这里的 80% 是课堂规定的阈值，不是通用故障标准。>= 包含边界 80；> 不包含。边界差异正是自动化测试应验证的地方。

and 要求两个条件都成立；or 至少一个成立；not 取反。区分赋值 = 和比较 ==，判断是否为 None 使用 is None。不要拿 is 比较普通字符串或数字内容。

循中　continue 跳过当前一项，break 结束整个循环。采集一个进程失败通常应跳过这一项，而不是放弃所有进程。无限循环必须有停止或超时机制，本周采集器只做一次快照。[S09]

03.4 函数、参数、返回值与契约【必会】

函数把一项任务变成可复用的“输入 → 输出”。你在调用前应知道它接收什么，成功返回什么，失败抛出什么。这叫函数契约，不要求使用额外框架。

```
def utilization(used: float, total: float) -> float:
  if total <= 0:
    raise ValueError("total must be positive")
  if used < 0 or used > total:
    raise ValueError("used must be between 0 and total")
  return round(used / total * 100, 2)

result = utilization(3, 8)
print(result)          # 37.5
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_04.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_04.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

python chapter_code/code_04.py

def 定义函数；括号中是参数；冒号后缩进部分是函数体。先验证 total 再相除，避免除零。raise 把无效输入作为错误反馈，下一章会解释如何接住。return 返回给调用者并结束本次执行；print 只是输出文本。

如果把最后一行改成 print(...) 而没有 return，调用结果会变为 None。后续把结果写入 JSON 或交给另一个函数时，这种错误很隐蔽。**计算函数返回数据，入口程序负责展示，**是本周的默认分工。

上面的函数是帮助理解的数值示例，假定传入普通有限数值。面向外部输入时还需要类型、NaN、无穷大等校验；参考工程的阈值判断补齐这些情况，不把简化例子冒充完整校验器。

03.5 默认参数与作用域【必会】

```
def is_high_usage(percent: float, threshold: float = 80.0) -> bool:
    return percent >= threshold


print(is_high_usage(85.0))
print(is_high_usage(85.0, threshold=90.0))
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_05.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_05.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_05.py
```

默认阈值在未传入时使用；关键字参数让调用更清楚。上例依次是 True、False。默认参数不是每次调用重新计算，所以不要写 def add(x, items=[]) 后不断修改这个列表；不同调用会共享它。

```
def add_item(value, items=None):
    if items is None:
        items = []
    items.append(value)
    return items
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_06.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_06.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_06.py
```

函数里创建的名字通常属于局部作用域。读取名字时，初学阶段可以按“局部 → 外层函数 → 模块全局 → 内置”理解；这常称为 LEGB。函数内赋值通常创建局部绑定，不会自动修改模块里的同名变量。[S09、S11]

```
threshold = 80

def local_demo():
    threshold = 90
    return threshold
```

```
print(local_demo(), threshold)   # 90 80
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_07.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_07.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_07.py
```

本周优先用参数与返回值传数据，不用大量 global 变量绕过作用域。这样测试一个函数时才不必猜它依赖哪些隐藏状态。

03.6 类型注解是说明，不是自动校验【必会】

percent: float 声明预期参数类型；-> bool 声明预期返回类型。list[float] 表示浮点数列表，str | None 表示字符串或 None，dict[str, object] 表示字符串键与一般对象值。

Python 运行时通常不会仅凭这些注解拒绝错误类型。def echo(x: int): return x 仍可能接收字符串。IDE、静态检查器可以利用注解提前提示，但外部输入是否符合范围仍需显式验证。[S12]

本周不要求学会高级泛型。你要能解释自己的参数与返回值，并在入口校验。类型注解不等于测试，也不等于安全边界。

03.7 排序、切片与最小复杂度意识【理解】

采集器要找内存最高的 N 个进程，步骤是“收集 → 按 RSS 排序 → 取前 N 个”。

```
rows = [{"pid": 1, "rss": 20}, {"pid": 2, "rss": 50}]
ordered = sorted(rows, key=lambda item: item["rss"], reverse=True)
print(ordered[:1])       # [{"pid": 2, "rss": 50}]
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_08.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_08.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_08.py
```

lambda 是小型匿名函数，告诉 sorted 用哪个字段作排序依据；reverse=True 从大到小；[:1] 取前一项。sorted 返回新列表，list.sort() 原地修改且返回 None。

扫描 n 个记录通常需要 O(n) 工作量，完整排序通常按 O(n log n) 理解。本周不做算法题专项，但要知道进程数变多会增加采集时间。不要先把字节数四舍五入再排序，否则相近值可能产生人为并列。[S10]

本章练习 E03： 自己实现 classify_usage(percent, threshold=80)：合法输入返回 normal 或 high；percent 与 threshold 都必须是 0~100 的有限数；无效输入抛出 ValueError。测试 79.9、80、100、-1、101、NaN。先写规则，再看 lab/monitor/rules.py。

04 | 文件、JSON 与资源清理

本章摘要： 字典只在当前程序内存在，JSON 让它成为可保存、可交换的记录。正确关闭文件和明确数据格式，比“文件生成了”更重要。

04.1 路径对象与当前工作目录【必会】

pathlib.Path 是路径操作接口，不会自动创建文件，也不保证路径存在。Path('reports/a.json') 是一个路径对象，直到实际写入才访问文件系统。

```
from pathlib import Path

path = Path("reports/first.json")
print(Path.cwd())
print(path.resolve())
path.parent.mkdir(parents=True, exist_ok=True)
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_09.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_09.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_09.py
```

.parent 是父目录，parents=True 创建缺失的多级父目录，exist_ok=True 允许已存在。这不意味着任意错误都会被忽略；若路径中某一段其实是普通文件，仍会失败。

找不到文件时，先检查 Path.cwd() 和绝对路径，再检查拼写、大小写和权限，不要直接把“路径错误”判断成“Python 库坏了”。[S13]

04.2 文本、字节与编码【必会】

文本文件存储的底层仍是字节，编码规定字符怎样映射成字节。本周文本统一使用 UTF-8，读写都显式写 encoding='utf-8'。图片和模型权重通常是二进制数据，不应该用普通文本方式读写。

'r' 读取；'w' 写入并覆盖已有内容；'a' 追加；'rb'/'wb' 是二进制方式。'w' 的覆盖发生在打开文件时，不能指望“后面写入报错，旧文件一定还在”。[S14]

```
from pathlib import Path

path = Path("scratch/utf8.txt")
path.parent.mkdir(parents=True, exist_ok=True)
```

```
with path.open("w", encoding="utf-8") as handle:
    handle.write("系统采集正常\n")
with path.open("r", encoding="utf-8") as handle:
    print(handle.read())
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_10.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_10.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_10.py
```

04.3 with 为什么能减少资源泄漏【必会】

打开文件通常会占用操作系统资源。with 使用上下文管理协议，在代码块离开时执行清理；正常完成或发生常规异常都会走退出逻辑。它不是“忽略所有异常”的语法，也不等于数据已经得到断电级持久化保证。[S14、S15]

相比手动 open → 操作 → close，with 能减少中间异常导致忘记关闭的风险。进程被强制杀死等特殊情况不能按普通 Python 控制流理解；第一周先掌握正常与异常退出时的管理方式。

04.4 JSON 和 Python 字典有什么区别【必会】

字典是 Python 内存对象；JSON 是一种文本数据格式。JSON 对象的键必须是字符串；字符串通常用双引号；布尔值写作 true/false，空值写作 null。Python 中对应 True/False/None。

```
import json

report = {"cpu_percent": 25.5, "ok": True, "gpu": None}
text = json.dumps(report, ensure_ascii=False, allow_nan=False)
restored = json.loads(text)
print(text)
print(restored == report)   # True
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_11.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_11.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_11.py
```

dumps/loads 在对象与字符串之间转换；dump/load 直接对文件对象操作。日期对象、Path、集合、任意类实例不能直接当作普通 JSON 值，应先转成字符串、列表等明确格式。allow_nan=False 防止把 NaN/Infinity 当作可互操作的标准 JSON 数字写出。[S16]

04.5 完成一次保存—读取闭环【必会】

```
import json
from pathlib import Path

def save_json(data: dict, path: Path) -> None:
    text = json.dumps(data, ensure_ascii=False,
                      indent=2, allow_nan=False)
    path.parent.mkdir(parents=True, exist_ok=True)
    with path.open("w", encoding="utf-8") as handle:
        handle.write(text + "\n")

def read_json(path: Path) -> dict:
    with path.open("r", encoding="utf-8") as handle:
        return json.load(handle)

source = {"cpu_percent": 25.5, "note": "首次采集"}
target = Path("reports/first.json")
save_json(source, target)
assert read_json(target) == source
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_12.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_12.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_12.py
```

先 dumps 再打开文件，能避免因为对象无法序列化而先清空旧报告。不过这仍不是原子写入：写到一半发生磁盘故障或进程中断，文件仍可能不完整。参考工程明确保留这个入门版限制；原子替换和持久化语义留到后续可靠性课程。

本章练习 E04： 保存含中文、列表、布尔值和 None 的报告，再读出比较。把文件手工改成缺少引号的 JSON，观察错误类型。再把“某个应是目录的路径”创建成普通文件，观察为什么 mkdir 失败。只能操作 scratch 下的实验数据。

05 | 模块、包与类：让代码能够增长

本章摘要： 模块解决“代码放哪”，类解决“状态与行为如何放在一起”，入口解决“从哪里启动”。不是项目越小越要套复杂设计。

05.1 单文件什么时候该拆【必会】

当一个脚本同时负责采集系统、写文件、处理参数、输出日志时，你很难单独测试“存储是否正确”。拆分的判断标准是职责，而不是凑文件数量。

本周采用以下结构，所有运行命令从 lab/ 或你自己的同结构项目根目录执行：

```
lab/
  monitor/
    __init__.py
    __main__.py
    collector.py
    storage.py
    rules.py
  tests/
  reports/
  logs/
  requirements.txt
  pyproject.toml
  .gitignore
  README.md
```

collector.py 读取系统；rules.py 放无系统依赖的判断规则；storage.py 读写报告；__main__.py 组织命令行执行。测试文件与业务文件分开，运行产物与源码分开。

05.2 import、模块与包【必会】

通常一个 .py 文件可作为模块；本项目用含 __init__.py 的目录作为普通包。from monitor.rules import classify_usage 的意思是从这个模块取出函数定义，不是把整段源码复制到当前文件。[S17]

导入时模块顶层代码会执行，因此不要把耗时采样、删文件或交互输入直接写在可导入模块的顶层。把动作放在函数里，由入口显式调用。另一个常见错误是把文件命名为 json.py 或 psutil.py，遮蔽了真正的库。

模块找不到时，先看当前目录、解释器和文件名。本课程从项目根目录运行，不用四处修改 sys.path 来掩盖结构问题。

05.3 main、__name__ 与 python -m【必会】

```python
def main() -> int:
    print("monitor started")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_13.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_13.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

python chapter_code/code_13.py

文件作为启动脚本运行时，__name__ 是 __main__；被导入时通常是模块名。这个判断把“可复用定义”和“直接执行动作”分开。SystemExit(main()) 把 main 返回值传为进程退出状态，不是异常打印的普通报错。[S17]

python -m monitor 表示让 Python 找到包 monitor 并执行其 __main__.py。不要进入 monitor/ 内再运行这个命令，因为那样的导入查找起点不同。本课程不要求先把项目发布到 PyPI。

05.4 类、对象、属性和方法【必会基础】

函数适合无长期状态的计算；类适合把一组配置和相关操作放在一起。例如采集器每次都需要磁盘路径、采样间隔和进程数量，这些可以保存到对象里。

```
class MonitorConfig:
    def __init__(self, interval: float = 0.5):
        if not 0 < interval <= 10:
            raise ValueError("interval out of range")
        self.interval = interval

    def describe(self) -> str:
        return f"sample interval: {self.interval}s"

config = MonitorConfig(1.0)
print(config.describe())
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_14.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_14.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

python chapter_code/code_14.py

类是创建这种对象的定义；config 是实例。__init__ 负责初始化已经创建的实例；self 表示当前实例，调用方法时由 Python 传入。self.interval 是实例属性；describe 是方法。不要把 self 理解成固定的全局变量。[S11]

两个实例可以保存不同间隔，互不干扰。把可变列表放在类属性上可能使实例共享数据；本周把每个对象自己的可变状态放在 __init__ 中创建。简单的 JSON 保存函数不必为了“面向对象”强行变成复杂类。

05.5 接口、私有约定与耦合【理解】

参考采集器的 collect() 返回字典；调用者不需要知道每一个系统调用细节。_top_processes() 的前导下划线表示内部使用约定，不是无法访问的安全限制。接口的价值是让入口程序依赖稳定的行为，而不是依赖内部实现。

本周不要求继承、设计模式、元类。你需要能指出哪一层负责什么，并能单独替换其中一层。例如测试存储时传一个固定字典，不应该必须真实读取 CPU。

本章练习 E05： 把自己的单文件脚本拆为采集、存储、入口三部分。先导入采集模块，确认不会立刻生成报告；再执行入口生成报告。能解释这两个动作的差别才算完成。

06｜异常与日志：失败也要有明确行为

本章摘要： 异常回答“接下来怎么处理”，日志回答“发生了什么”。既不能遇错全停，也不能把所有错误悄悄吞掉。

06.1 语法错误、运行异常与逻辑错误【必会】

语法错误例如缺少冒号，程序尚未按预期开始执行；运行异常例如文件不存在、除零；逻辑错误是程序跑完却给出错误答案，例如把阈值边界 >=80 写成 >80。测试能发现一部分逻辑错误，日志本身不能证明计算正确。[S18]

常见错误：NameError 表示名字未定义，TypeError 表示类型/操作不匹配，ValueError 表示值不符合要求，FileNotFoundError 表示路径不存在，PermissionError 表示权限不足，JSONDecodeError 表示 JSON 解析失败。

阅读 traceback 时先看最末行的异常类型和消息，再向上找自己的代码行，接着核对当时参数、路径与环境。不要只截取最后一句英文去盲搜，也不要看见库文件栈帧就断定库有 bug。

06.2 try、except、else、finally【必会】

```
from pathlib import Path

try:
    content = Path("scratch/config.txt").read_text(encoding="utf-8")
except FileNotFoundError:
    print("config missing; use classroom default")
else:
    print("read succeeded:", content)
finally:
    print("this attempt has ended")
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_15.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_15.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_15.py
```

try 包住可能失败的动作；except 只处理匹配异常；else 在 try 没发生异常时执行；finally 在离开此结构时执行清理或收尾。finally 中不要随意 return，它可能覆盖原返回值或掩盖异常。

只有“文件缺失可以使用默认配置”这个业务约定成立时，才应这样恢复。缺少关键模型文件却继续假装推理成功，就是错误降级。异常策略必须围绕任务语义，而不是围绕“不想看见报错”。[S18]

06.3 三类失败如何决定【必会】

情况	本周策略	原因
一个进程在采集时退出或不可读	记录警告、跳过、标记列表不完整	其他进程仍可读取
用户给出非法 interval/top/threshold	明确拒绝，不进行正常采集	输入契约不满足
整个报告无法保存	返回失败状态并保留错误线索	核心交付未完成

进程列表与逐个读取之间存在时间差，所以“刚看到 PID，读取时已退出”是正常竞争条件。它不是一定需要 root 解决的问题。读取不了也不能写假数补齐。

except Exception: pass 会丢失所有证据，也可能掩盖真正代码错误。本周只捕获预期异常。意外的 AttributeError 等通常应让测试暴露，不应为了显示“全部成功”而吞掉。

06.4 日志级别与日志内容【必会】

DEBUG 用于细节诊断，INFO 用于正常关键事件，WARNING 表示出现需要注意但仍可继续的情况，ERROR 表示某项操作失败，CRITICAL 表示严重到可能无法继续的情况。级别是你赋予事件的意义，不是 logging 自动替你判断严重程度。[S19]

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s %(message)s",
)
logger = logging.getLogger(__name__)
logger.info("sampling started; interval=%s", 0.5)
try:
    raise OSError("classroom simulated failure")
except OSError:
    logger.exception("report could not be saved")
```

代码操作（补充）： 本例已提供完整源码 chapter_code/code_16.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_16.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_16.py
```

预期在终端错误通道看到 INFO、ERROR 和异常栈。logger.exception 应在处理异常时调用，用于附带异常信息。日志不是只能写文件；handler 决定输出到终端、文件或其他目的地，formatter 决定格式。[S19、S20]

一条有用日志至少说明时间、模块、动作、必要参数和结果。例如“保存报告失败，目标路径是哪里”，比“发生错误”更有价值。不要记录密码、密钥、完整私人命令行或不必要的个人信息。

06.5 日志文件本身也可能失败【必会】

创建 logs/、打开日志文件，都可能遇到权限或路径问题。不能把日志初始化放在错误处理之外，然后承诺“任何失败都有日志”。参考入口会在日志不可用时至少把错误写到 stderr，并返回非零状态。

同一进程反复配置日志还可能重复添加 handler，导致一条消息输出多次。参考工程集中在入口配置并清理自己的 handler，业务模块只获取 logger，不在每个函数里重新配置。

06.6 错误信息与退出状态需要一致【必会】

本项目约定：成功完成核心采集和写入返回 0；可预期运行失败返回 1；argparse 发现命令格式或参数类型错误通常返回 2。进程列表部分不可读可继续返回 0，但 JSON 必须标明不完整，不能谎称收集到了所有进程。

同样返回 0 也不表示“硬件健康”，只表示这个命令按约定完成。健康结论需要后续持续监控、上下文和业务指标。

本章练习 E06： 制作三份记录：非法阈值、报告保存失败、单进程不可读。每份写清“继续/退出、输出通道、退出状态、证据”。单进程不可读用测试模拟，不要去破坏真实进程权限。

07 | 进程、资源指标与硬件：从读数到判断

本章摘要： 会打印 CPU 和内存，只完成了“取数”。还要解释它们的采样范围、单位和局限，才能把它们作为诊断证据。

07.1 程序、进程、线程【必会】

程序是代码或可执行文件；进程是它运行中的实例。同一份 Python 脚本运行两次，通常会产生两个进程。PID 是进程标识符，PPID 是父进程标识符；PID 会被重复使用，不是永久身份。

进程拥有自己的虚拟地址空间等资源；线程是进程内的执行路径，通常共享该进程的地址空间和文件等资源。共享方便协作，也带来同时修改数据的竞争问题。线程不等于一个独立 CPU 核，操作系统负责把可运行线程调度到可用 CPU。

并发表示多个任务交替推进；并行表示某个时刻同时执行。多个线程不保证 Python 代码按核心数等比例加速；常规启用 GIL 的 CPython 中，纯 Python 计算会受其限制，而 IO 等

待与会释放 GIL 的库又不同。不要把这个结论推广成“所有 Python 版本、构建和扩展都绝对无法多核”。本周只理解差异，不编写并发框架。[S21、S22]

07.2 一个程序从启动到结束【必会】

在 Shell 输入命令后，Shell 解析参数并定位可执行程序；执行环境启动 Python；Python 加载代码与模块；线程得到调度后执行任务，可能等待 IO；正常结束或收到终止信号后进程退出，父进程可获取退出状态。实际系统细节比这个教学模型更复杂，但已足够解释本周故障。

一个任务等待磁盘或网络时，未必占满 CPU；所以 CPU 很低也可能响应很慢。进程状态中 R 常表示运行或可运行，S 常表示可中断睡眠，D 常与不可中断等待有关，Z 表示已退出但尚未被父进程回收的状态。第一周只做观察，不尝试处理系统进程。[S03]

07.3 安全观察自己启动的进程【必会】

```
sleep 30 &
lab_pid=$!
ps -p "$lab_pid" -o pid,ppid,stat,comm
kill -TERM "$lab_pid"
wait "$lab_pid"
echo $?
```

& 将本命令放到后台；$! 是最近后台任务的 PID；ps -p 只查指定进程；TERM 请求终止；wait 等待并回收这个由当前 Shell 启动的任务。若动作在 30 秒内完成，在 Bash 中通常会看到与 TERM 对应的非零状态；如果任务已自然结束，结果会不同。[S23]

Ctrl+C 通常向前台进程组发送中断信号。不要默认使用 kill -9，它不给程序普通清理机会；本实验也不需要它。只使用刚保存的自己的 PID，不照抄别人的 PID，更不要终止共享机器任务。

07.4 CPU：利用率不是算力百分比【必会】

CPU 使用率描述采样区间内的忙碌程度，不是芯片“已消耗了多少寿命”，也不直接等于模型吞吐率。高利用率可能是有效计算，也可能是低效忙循环；低利用率可能是任务轻，也可能在等 IO。

```
import psutil

print(psutil.cpu_count(logical=True))
print(psutil.cpu_percent(interval=1.0))
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_17.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_17.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_17.py
```

本例依赖 psutil，请先在已激活的虚拟环境执行 python -m pip install -r lab/requirements.txt（或在 lab/ 目录运行安装命令）。

interval=1.0 会阻塞大约一个采样窗口，再返回系统总体百分比。非阻塞的首次 cpu_percent(interval=None) 缺少前一有效基线，不能直接当作可靠测量；本周用正数 interval 避免这个陷阱。[S21]

本项目取的是全系统总体值，范围通常为 0~100。单进程 CPU 百分比在某些工具/统计口径下可超过 100，多核统计的归一化方式不同，不可直接比较。逻辑 CPU 数也不等于物理核心数，更不一定等于容器实际获准使用的 CPU 配额。[S21]

Linux 上可观察 lscpu、top；退出 top 用 q。ps 的 %CPU 和 top 的采样口径可能不同。负载均值也不是 CPU 百分比，它还涉及可运行任务或特定等待状态；本周不依据单个数字判定瓶颈。

07.5 内存：total、free、available、RSS【必会】

total 是环境可见总内存；free 是完全空闲的部分；available 估计可供新任务使用、通常不需交换的内存，其中可包含可回收缓存。Linux 会利用部分闲置内存缓存文件，因此 free 小不自动代表“内存快用尽”。

psutil.virtual_memory().percent 按 total 与 available 计算其使用比例。不要手工拿 1-free/total 就声称与它完全一致。先记录 total、available 和 percent，结合趋势观察。[S21]

```
import psutil

memory = psutil.virtual_memory()
print("total MiB:", memory.total / 1024**2)
print("available MiB:", memory.available / 1024**2)
print("percent:", memory.percent)
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_18.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_18.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_18.py
```

本例依赖 psutil，请先在已激活的虚拟环境执行 python -m pip install -r lab/requirements.txt（或在 lab/ 目录运行安装命令）。

MiB = 1024^2 字节，GiB = 1024^3 字节；MB、GB 通常按十进制定义。报告字段写 _mib 或 _gib，避免用 1024 计算却标成 MB。保留单位是工程习惯，不是格式细节。

RSS 是进程当前驻留物理内存的统计量；VMS 是其虚拟地址空间规模。VMS 很大不代表占用了同等物理内存。RSS 可以包含共享页，因此把所有进程 RSS 简单相加，不一定等于系统已用内存。[S21、S03]

07.6 内存增长、资源耗尽与泄漏【必会判断边界】

存累、据增加、存分配器保留空、正的象引用泄漏，都可能造成存
增长。两次 RSS 增加只能证明“观察到了增长”，不能直接证明泄漏。

诊断应控制输入与请求量，重复相同工作负载，观察停止负载后是否趋于稳定，再结合对象、文件句柄、连接等数据深入分析。文件没有关闭可能耗尽文件描述符；这与 Python 对象占用过多内存不是同一种资源问题。with 能帮助清理文件，但不会自动解决所有内存问题。

本周不通过耗尽内存来制造故障，不运行无限分配脚本。使用测试模拟失败，或对正常轻量程序做观察，足够学习基本方法。

07.7 磁盘：空间使用率不等于 IO 性能【必会】

psutil.disk_usage(path) 查询的是该路径所在文件系统的空间使用信息。disk_usage('/') 不一定代表机器所有物理磁盘之和；WSL 或容器中的挂载与宿主机也可能不同。[S21]

```
free -h
df -h .
ps -eo pid,ppid,stat,rss,comm --sort=-rss | head -n 10
```

df -h . 看当前目录所属文件系统；ps 这里的 rss 在 Linux 常用 KiB 表示，而程序以字节换算 MiB，所以比较前先统一单位。磁盘占用 30% 不代表读写很快；空间容量与 IO 吞吐、延迟是不同指标。

07.8 CPU/GPU/NPU 与 x86/ARM【理解】

CPU 偏通用控制与计算；GPU 强调大规模并行吞吐；NPU 是面向神经网络计算的专用加速类别。不是“GPU 必然比 CPU 快”：任务规模、数据传输、算子支持和软件实现都影响结果。

x86-64、ARM64 是指令集架构类别，不是与 GPU/NPU 平级的“快慢等级”。以后相同程序的本地扩展包、驱动和部署镜像可能需要匹配架构。本周只要求看懂 uname -m 的结果，不要求学汇编。

USB 是设备连接接口，UART 是常见串口通信方式，I^2C/SPI 常用于板级器件通信。第一周知道这些名词与“数据如何进入系统”有关即可，实际接线和协议实验留到后续设备课程。不要为了完成本周学习购买一整套硬件。

lsusb、lspci 可能需要额外工具包并受环境限制；nvidia-smi 还要求兼容的 NVIDIA 硬件及驱动环境。命令不存在并不能证明硬件不存在。本周可不安装这三类工具，不为观察信息而修改驱动。

07.9 时间、采样与可比性【必会】

报告时间戳回答“何时发生”，耗时回答“持续多久”。时间戳可用带时区的 UTC 时间；测量区间耗时优先使用 time.perf_counter() 等适合测间隔的时钟。墙上时间可能受校时调整，不应随意拿来做精细耗时比较。[S24]

CPU 在窗口中统计，内存和磁盘在随后查询，进程又是逐一读取，所以一份 JSON 是接近同一时段的快照组合，不是所有指标在同一个原子时刻的状态。参考报告同时保存采样起止时间和 CPU 窗口配置。

做对比至少固定系统环境、采样间隔和观察动作，并记录并发活动。单次采样不能证明长期稳定性，更不能从“开了浏览器后 CPU 增加”直接断定某个网页是唯一原因。

本章练习 E07： 空闲状态下用同一间隔连续采集 3 次；打开一个平时使用的轻量程序后再采集 3 次。描述事实、可能原因和还缺的证据。不要要求六个读数完全一致，也不要以数值高低给自己的电脑打分。

08｜生成器、装饰器、上下文管理器

本章摘要： 三种机制分别解决“按需生成数据”“给函数加一层行为”“管理进入与退出”。本周能解释和改例子即可，不要求设计框架。

08.1 生成器与 yield【理解】

列表通常已经保存好一组元素；生成器按请求继续执行并产出下一项。含 yield 的函数在调用时返回生成器，执行会在迭代时推进。yield 产出一个值后暂停，return 则结束本次生成。[S11]

```
def high_values(values, threshold=80):
    for value in values:
        if value >= threshold:
            yield value

result = high_values([20, 80, 95])
print(next(result))      # 80
print(list(result))      # [95]
print(list(result))      # []
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_19.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_19.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_19.py
```

第一个 next 消耗 80，随后 list 只剩 95，最后已经耗尽。这说明生成器不是每次自动从头开始的列表；要再遍历，应重新调用函数创建生成器。

读取大日志时逐行处理可以减少一次性载入的内存，但不是所有“使用 yield 的代码”都节省内存：如果先在函数内部读取全部文件，仍然占用那部分空间。文件类生成器未耗尽时可能仍保持打开的资源，因此要明确关闭时机，不把生成器当作免费资源管理器。

08.2 装饰器：输入函数，返回包装函数【理解】

```
import time
from functools import wraps

def timed(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            elapsed = time.perf_counter() - start
            print(f"{func.__name__}: {elapsed:.6f}s")
    return wrapper

@timed
def sum_values(limit):
    return sum(range(limit))

print(sum_values(1000))    # 499500，另有一行实际耗时
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_20.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_20.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_20.py
```

@timed 可近似理解成 sum_values = timed(sum_values)。wrapper 在原函数前后加入计时。*args 接收位置参数，**kwargs 接收关键字参数，再原样传给原函数。return func(...) 保留原返回值，漏写 return 会改变功能。

wraps 保留原函数名称等元信息，便于调试；finally 保证普通异常路径也记录耗时，但不吞掉原异常。这只是教学计时器，不是严谨性能基准。第一次加载、系统噪声、缓存、GPU 异步执行等，都会让后续性能测量更复杂。[S24、S25]

08.3 上下文管理器：进入、使用、离开【理解】

你已经用过 with file.open(...)。更一般的上下文管理器通过进入和退出逻辑管理一个作用范围。下面例子只演示顺序，不要求本周自己封装数据库连接。

```
from contextlib import contextmanager

@contextmanager
def experiment(name):
    print("begin:", name)
```

```
    try:
        yield
    finally:
        print("end:", name)

with experiment("sample"):
    print("collecting")
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_21.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_21.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_21.py
```

预期顺序为 begin、collecting、end。@contextmanager 用单次 yield 把函数分成进入阶段与离开阶段；finally 放清理逻辑。上下文管理器可能选择处理异常，但本例不抑制异常，不应把它当成 try/except 的自动替代品。[S15]

本章练习 E08： 解释为什么生成器第二次转 list 是空列表；给一个返回 42 的函数加 timed，确认返回值仍是 42；在 with 的内部主动抛 ValueError，观察 end 是否输出以及错误是否仍然向外传播。

09｜测试工程：怎样证明功能符合约定

本章摘要： 测试不是再运行一遍程序，而是先定义预期，再用独立证据核对实际。必须覆盖正常、边界与错误路径。

09.1 测试层次与本周范围【必会】

单元测试聚焦小函数或模块，尽量隔离外部依赖；集成测试验证模块之间的组合；系统/端到端测试从真实入口走到结果；回归测试是修改后重跑已有测试以检查是否破坏功能，并不是与前三者互斥的层级。

本周核心是规则与存储的单元测试、采集模块的可控模拟，以及一次真实命令行采样。真实机器读数会变化，所以“运行正常”不能替代可重复的单元测试。

09.2 测试用例怎样写【必会】

先写清前置条件、输入、步骤、预期结果，再执行。不要看到输出后才临时决定“这个也算对”。

用例	输入	预期	为什么
正常低值	30，阈值 80	normal	典型正常范围
边界下方	79.9，阈值 80	normal	检查边界附近
正好边界	80，阈值 80	high	检查包含关系

用例	输入	预期	为什么
上界	100，阈值 80	high	合法最大值
非法范围	-1 或 101	ValueError	不接受异常百分比
非有限数	NaN、Infinity	ValueError	防止错误比较和 JSON 数据

等价类是把输入按预期行为分组，例如合法范围与非法范围；边界值是在分组交界处取样。你不是靠穷举所有浮点数测试，而是挑更有辨别力的代表。

09.3 pytest、assert 与命名【必会】

解压完整 ZIP 后，先进入 Week1_Learning_Kit_v4/lab/。在该目录中，monitor/ 与 tests/ 已经存在并且同级，tests/test_rules.py 也已提供。下面是简化案例；自己练习时请创建 tests/test_09_03_practice.py，不要覆盖已有文件。

```
from monitor.rules import classify_usage

def test_boundary_is_high():
    result = classify_usage(80.0, threshold=80.0)
    assert result == "high"
```

操作步骤（先做一次）：请在普通 Linux 终端从解压文件夹的上一级进入 Week1_Learning_Kit_v4/lab，使用 ls monitor tests 确认目录。安装 requirements.txt 后可运行现成测试 student_examples/test_09_03_boundary.py。若亲自输入，使用 nano tests/test_09_03_practice.py，保存后执行 python3 -m pytest -q tests/test_09_03_practice.py。不要覆盖已有 tests/test_rules.py。

```
cd Week1_Learning_Kit_v4/lab
pwd
ls monitor tests
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python3 -m pytest -q student_examples/test_09_03_boundary.py
python3 -m pytest -q tests/test_rules.py
```

注意：如果你已在 lab/ 中，不要再执行 cd。第 09 章的测试必须从 lab/ 运行；不要在 chapter_code/ 中测试依赖 monitor 的代码。

用 python3 -m pytest -q 从资料包 lab/ 目录运行。pytest 依据约定发现 test 开头的文件和函数；assert 表达预期，失败时会显示实际值差异。-q 减少输出，不改变测试本身。[S26]

assert 在测试中很有用，但不要只靠业务代码中的 assert 校验不可信输入，因为 Python 优化执行模式可能移除 assert。业务校验用明确的条件与 raise。

测试常用“准备 Arrange → 执行 Act → 断言 Assert”的顺序。一个测试尽量检验一个明确行为，不要把二十个无关断言塞进一个函数，导致失败后不知道哪项需求被破坏。

09.4 参数化与异常测试【必会】

```python
import pytest
from monitor.rules import classify_usage

@pytest.mark.parametrize("value, expected", [
    (79.9, "normal"),
    (80.0, "high"),
    (100.0, "high"),
])
def test_threshold_cases(value, expected):
    assert classify_usage(value) == expected

def test_invalid_percentage():
    with pytest.raises(ValueError):
        classify_usage(-1.0)
```

操作步骤（沿用 09.3 进入的 lab/ 工作目录）：本节参考代码在 student_examples/test_09_04_parameters.py。如想自己输入，运行 nano tests/test_09_04_practice.py，写完保存，再执行 python3 -m pytest -q tests/test_09_04_practice.py。monitor/ 和 tests/ 已在资料包里，不需要你从零创建。

```
python3 -m pytest -q student_examples/test_09_04_parameters.py
```

注意：此测试导入 monitor 包，须从 lab/ 目录运行，不可在 chapter_code/ 目录下直接执行。

参数化让相同逻辑接受多组数据，每组成为独立测试案例；不用手工复制三个相似函数。pytest.raises 的意思是这段操作应该抛出指定异常，没抛或抛错类型都失败，不是“报错也算随便通过”。[S27]

09.5 fixture 与 tmp_path【必会】

fixture 为测试提供所需条件和资源。测试函数参数名 tmp_path 由 pytest 注入，不是你要自己定义的全局路径。它给每个测试提供独立临时目录，避免污染正式报告。

```python
from monitor.storage import read_json, save_json

def test_json_round_trip(tmp_path):
    path = tmp_path / "nested" / "report.json"
    data = {"ok": True, "description": "测试"}
    save_json(data, path)
    assert path.is_file()
    assert read_json(path) == data
```

操作步骤（沿用 09.3 进入的 lab/ 工作目录）：本节参考代码在 student_examples/test_09_05_tmp_path.py。如想自己输入，运行 nano

tests/test_09_05_practice.py，写完保存，再执行 python3 -m pytest -q tests/test_09_05_practice.py。monitor/ 和 tests/ 已在资料包里，不需要你从零创建。

python3 -m pytest -q student_examples/test_09_05_tmp_path.py

注意：此测试导入 monitor 包，须从 lab/ 目录运行，不可在 chapter_code/ 目录下直接执行。

此测试同时观察文件创建和内容回读。如果只断言 path.exists()，一个空文件也能通过，证据不足。临时目录不是数据备份位置，重要实验结果仍需另存报告。[S28]

09.6 monkeypatch：控制外部条件，不造假结果【理解并完成一次】

系统资源一直在变，单元测试不能写“我的 CPU 必须等于 25%”。可以在测试期间替换采样函数，让返回值确定，再验证自己的处理逻辑。

```
from monitor import collector as module

def test_fixed_cpu_reading(monkeypatch):
    monkeypatch.setattr(
        module.psutil, "cpu_percent",
        lambda interval: 25.0,
    )
    assert module.psutil.cpu_percent(interval=0.1) == 25.0
```

操作步骤（沿用 09.3 进入的 lab/ 工作目录）：本节参考代码在 student_examples/test_09_06_monkeypatch.py。如想自己输入，运行 nano tests/test_09_06_practice.py，写完保存，再执行 python3 -m pytest -q tests/test_09_06_practice.py。monitor/ 和 tests/ 已在资料包里，不需要你从零创建。

python3 -m pytest -q student_examples/test_09_06_monkeypatch.py

注意：此测试导入 monitor 包，须从 lab/ 目录运行，不可在 chapter_code/ 目录下直接执行。

这个最小例子只展示替换行为；完整工程还会模拟内存、磁盘和进程，再测试 collect() 的输出。补丁在测试结束后由 fixture 恢复。应该替换被测代码实际查找的对象位置，而不是猜一个同名函数。[S29]

模拟测试证明的是“给定这些外部响应，我的逻辑如何处理”，不证明真实设备或驱动一定正常。故障注入测试与真实采样各有作用，不能互相冒充。

09.7 测试失败时怎样看【必会】

先看失败测试名，找到对应需求；再看 expected 与 actual，确认是实现错、预期错还是环境不满足；只修改有证据的问题；重跑当前测试与全部测试。不要为了全绿把所有断言删掉。

参考工程包含规则、JSON、采集、局部进程失败与 CLI 失败的测试。通过数量只表示当前用例通过，不等于无 bug，也不等于能胜任所有生产场景。

本章练习 E09： 不抄现有测试，自己新增至少五个：阈值等于 0、等于 100、非法阈值、中文 JSON 回读、错误输出路径。再故意将 >= 改为 >，确认测试能抓到；随后修复并回归。测试抓不到人为错误时，应检查用例覆盖。

10 | Git 与技术文档：让成果能够解释和复现

本章摘要： Git 保存代码演化，文档保存运行条件和判断证据。二者都不只是“最后提交作业时再补”。

10.1 工作区、暂存区、提交【必会】

工作区是你正在编辑的文件；暂存区记录准备进入下一次提交的内容；提交是带说明的版本快照。git add 不是上传，git commit 也不等于已经推到远程服务器。[S30]

在自己的项目根目录进行初始化。以下姓名和邮箱是演示值，应换成你准备使用的提交身份；本周只需本地仓库。

```
git init -b main
git config user.name "Student"
git config user.email "student@example.invalid"
git status
```

这些 config 不带 --global，只作用于此仓库。提交身份可能进入公开历史；不愿公开私人邮箱时使用合适的非私人提交身份。本课程不替你创建网络账号。

10.2 先写 .gitignore 再添加文件【必会】

```
.venv/
__pycache__/
.pytest_cache/
*.pyc
logs/*
!logs/.gitkeep
reports/*
!reports/.gitkeep
.env
```

.gitkeep 只是常见的空目录占位约定，不是 Git 内置特殊功能。原始日志与报告可能有个人信息，因此默认不提交；经过审查脱敏的周报可以放到 docs 下单独管理。

.gitignore 只阻止尚未跟踪的匹配文件自动加入，不会把已经提交的密钥从历史中清除。真正泄露的密钥需要撤销/轮换，而不是只加一行 ignore。[S30]

10.3 有意义的提交和差异检查【必会】

```
git status
git diff
git add monitor tests requirements.txt .gitignore README.md
git diff --cached
git commit -m "feat: add first system snapshot"
git log --oneline
```

git diff 看尚未暂存的改动；git diff --cached 看将要提交的改动。先检查，再提交。只提交存在且确认过的路径；项目尚未有 monitor/tests 时，先提交当时已有的文件，不要机械复制不存在的名字。

建议本周至少保留三次能解释的变化：实现最小采样、补充异常处理、增加测试。不要用大量“update”“final2”提交来替代清晰说明。

10.4 分支与合并【理解】

```
git switch -c feature/threshold
# edit, test, add and commit your change here
git switch main
git merge feature/threshold
```

分支让你在一个明确版本上推进一项改动，不是复制整个项目目录。先提交或妥善处理当前修改再切换，冲突发生时先理解两边改了什么。第一周不学复杂 rebase 和强制推送，也不使用 reset --hard 清理自己不理解的状态。[S31]

10.5 README 与故障记录应写什么【必会】

README 至少包含：项目解决的问题、支持环境、安装步骤、运行命令、参数说明、测试命令、一个输出结构、已知限制。它的检验方式是“另一个人能否从零照做”，不是“介绍看起来很专业”。

故障记录区分四类内容：**事实** 是命令、错误和测量；**假设** 是可能原因；**验证动作** 是如何排除其他可能；**结论** 是证据允许说到哪里。禁止把参考示例值填成自己的真实实验结果。

例如：事实是“输出路径父级是普通文件”；验证是 ls -l 和 Path.is_file()；结论是“路径结构不符合要求，无法创建子目录”。不能跳到“Linux 磁盘坏了”。

本章练习 E10： 让同学或一个新终端仅按你的 README 完成一次运行。记录缺失步骤，修改 README 并提交。没有同学也可以在新的工作目录建立虚拟环境检验步骤；不要删除原项目来模拟重装。

11 | 综合项目：Linux 系统信息采集器

本章摘要： 把前面知识串成一条可解释的执行链。先独立实现最小版本，再比对参考工程，最后做故障注入和交付。

11.1 需求与非目标【必会】

需求：一次命令采集 CPU、内存、磁盘及高 RSS 进程；把结果保存为 UTF-8 JSON；用参数调整采样间隔、数量和阈值；记录关键日志；跳过局部不可读进程并注明；核心操作失败返回非零退出状态；保留测试与复现说明。

非目标：这不是持续运行的生产监控系统，不包含数据库、GPU 监控、自动修复、网络诊断或模型推理。它没有完整访问控制、原子写入、日志轮转、并发执行锁和断电恢复保证。这些限制应写进 README，而不是用“企业级”掩盖。

11.2 第一步：单文件最小版本【Day 2 完成】

先不看完整参考工程，把下面代码保存为 first_monitor.py。它只演示采集与保存，尚未具有完整错误处理。

```
import json
from datetime import datetime, timezone
from pathlib import Path
import psutil

cpu = psutil.cpu_percent(interval=0.5)
memory = psutil.virtual_memory()
disk = psutil.disk_usage("/")
report = {
    "timestamp_utc": datetime.now(timezone.utc).isoformat(),
    "cpu_percent": cpu,
    "memory_percent": memory.percent,
    "disk_percent": disk.percent,
}
path = Path("reports/first_report.json")
path.parent.mkdir(parents=True, exist_ok=True)
with path.open("w", encoding="utf-8") as handle:
    json.dump(report, handle, indent=2, allow_nan=False)
print(path)
```

代码操作（补充）：本例已提供完整源码 chapter_code/code_26.py。在项目目录的 Linux 终端执行下面的命令；如要自己敲写，先执行 mkdir -p chapter_code（先建目录），然后执行 nano chapter_code/code_26.py，输入上面的代码，按 Ctrl+O、回车、Ctrl+X 保存后再运行。

```
python chapter_code/code_26.py
```

本例依赖 psutil，请先在已激活的虚拟环境执行 python -m pip install -r lab/requirements.txt（或在 lab/ 目录运行安装命令）。

```
python first_monitor.py
python -m json.tool reports/first_report.json
```

第一行采样会等待约 0.5 秒，后续查询再继续。datetime.now(timezone.utc) 生成带 UTC 时区信息的时间；isoformat() 转成 JSON 可存储的字符串。json.tool 用于验证和格式化已有 JSON，不是重新采集。

先预测报告应有哪些字段，再运行；修改 interval 后观察运行等待有何变化。不要预测具体 CPU 百分比，因为它来自你的实际环境。

11.3 第二步：按职责逐步改写【Day 3~5 完成】

先抽出 save_json/read_json，用固定字典测通；再把采集放进 SystemCollector.collect()；接着增加 rules.py 的阈值判断；最后添加命令行参数、日志、异常和测试。每一步都有可运行版本，不要一口气复制所有文件后再排几十个错误。

阶段	新增能力	继续前必须确认
V0	单文件采样与保存	JSON 能解析，字段含义说得清
V1	函数与模块	导入模块不产生采样副作用
V2	参数、类、日志与异常	非法输入被拒绝，日志位置明确
V3	进程列表与不完整标记	局部失败不伪造数据
V4	自动化测试与说明	正常、边界、异常都能验证

参考工程可以直接运行，但你的练习目录应保留自己的实现。不要覆盖自己尚未理解的代码，也不要把参考项目的测试通过当作自己已会写测试。

11.4 参考工程逐文件阅读顺序

先看 rules.py：它没有系统访问，只回答百分比是否合法、阈值是否达到。再看 storage.py：只负责对象与文件。第三看 collector.py：把外部系统读数转换为明确报告。最后看 __main__.py：负责参数、日志配置、调用与退出状态。测试按同样顺序阅读。

采集器构造函数校验 interval、top、cpu_threshold，并保存配置；collect 顺序调用 psutil；_top_processes 逐个读取 RSS 和名称，按原始字节数排序；返回前截取 top 条。遇到进程消失、访问被拒绝等预期情况，计数并写警告。

process_collection.incomplete=true 表示观察到了跳过项；false 只表示这次没有记录到此类跳过，**不保证看到完整物理主机的全部进程**。操作系统隔离、可见范围和采样竞态仍然存在。scope 字段说明这个范围。

报告中的 cpu_state 只应用课堂阈值规则，不能称为“AI 判断”或“硬件故障确认”。告警策略与客观读数分开保存，将来可以修改策略而不改变原始指标。

11.5 命令行参数与 argparse【必会基础】

命令行参数让用户改变行为，不必编辑源码。argparse 读取参数、转换指定类型并生成帮助；类型正确不等于业务范围正确，所以 interval 仍需在构造函数校验。[S32]

```
python -m monitor --help
python -m monitor --interval 0.5 --top 5 --cpu-threshold 80
python -m monitor --output reports/run02.json --disk-path .
```

参数	默认值	作用与约束
--interval	0.5	CPU 采样秒数，有限且大于 0、不超过 10
--top	5	返回前 N 个进程，整数 1~100
--cpu-threshold	80	课堂规则阈值，0~100 有限数
--disk-path	/	查询哪个路径所属文件系统
--output	reports/system_report.json	JSON 输出位置
--log-file	logs/monitor.log	应用日志位置，不能与输出相同

选择 interval 上限 10、top 上限 100，是本教材的工程契约，不是 psutil 的普遍限制。你以后可以改契约，但必须同步修改文档和测试。

参考入口把正常摘要写 stdout、失败提示写 stderr；日志能打开则写文件。日志初始化也在错误边界内。--top five 这种不能转成整数的参数通常由 argparse 返回 2；--interval 0 类型正确但范围错误，由本项目返回 1。

11.6 在参考工程中首次运行

解压完整资料包后找到 lab/，用文件管理器的“在终端打开”或 cd 进入它。你不需要假设它位于固定下载路径。确认 ls 能看到 monitor、tests、requirements.txt，再执行：

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pytest -q
python -m monitor --interval 0.5 --top 5
echo $?
python -m json.tool reports/system_report.json
tail -n 10 logs/monitor.log
```

预期：测试显示通过；采集命令打印报告位置与资源摘要；紧随其后的状态为 0；JSON 可解析；日志包含开始和保存事件。具体百分比、PID、进程名称必须以你的机器为准。

测试环境与实际结果在资料包 VALIDATION.txt 中记录。WSL、你的系统包安装、USB/PCI 枚举和 SSH 连接没有在你的设备上代测；这些不能被本机代码验证替代。

11.7 报告字段怎样读

字段	意义	不应该得出的结论
sample_started_utc / timestamp_utc	采集起止附近的 UTC 时间	所有读数原子同时取得
host.architecture	当前环境机器架构标识	可直接推断真实设备全部规格
metrics.cpu_percent	配置窗口内的 CPU 总体使用率	它就是模型性能或硬件健康
memory_available_mib	估计可用内存，单位 MiB	与完全空闲内存相同
disk_total_gib	指定路径所在文件系统容量	全机所有磁盘总容量
top_processes_by_rss	可读取记录中按 RSS 排序的前 N 项	RSS 总和等于系统内存
skipped_count	此次显式跳过的进程数量	没跳过就看到了整个宿主机

报告会暴露进程信息；提交作业或公开仓库前审查并脱敏，不要把真实个人运行环境原样发到公共平台。

11.8 五个故障实验【Day 6 必做】

只在自己的 lab 或练习项目下操作。每次执行完故障命令立即 echo $?，不要先运行 tail。不要使用 root 来“解决”本来应该被正确报告的失败。

F1：非法间隔。

```
python -m monitor --interval 0
echo $?
```

预期非零，本参考实现为 1，消息说明 interval 不符合约定。不要把它改成 0 后假装采样成功；0 窗口还会引入 CPU 测量语义问题。

F2：不存在的磁盘路径。

```
python -m monitor --disk-path /week1_nonexistent_path
echo $?
```

预期失败且有路径相关证据。先用 ls 验证这个路径确实不存在；如果你的环境碰巧有同名目录，换一个自己确认不存在的路径。

F3：输出父级被普通文件占据。

```
mkdir -p scratch
printf 'not a directory\n' > scratch/output_blocker
python -m monitor --output scratch/output_blocker/report.json
echo $?
```

预期失败；确认 blocker 是普通文件而非目录。这比依赖 chmod 模拟权限失败更可重复，因为管理员账户可能绕过部分权限限制。

F4：日志父级被普通文件占据。

```
printf 'not a directory\n' > scratch/log_blocker
python -m monitor --log-file scratch/log_blocker/monitor.log
echo $?
```

预期失败提示仍出现在 stderr，而不是要求“坏掉的日志系统必须记录自己”。理解“有控制地失败”也是正确行为。

F5：进程读取失败。

```
python -m pytest tests/test_collector.py -q
```

阅读 test_order_and_partial_failure 与 test_process_disappears，确认用模拟对象主动产生 AccessDenied/NoSuchProcess。它们检验跳过行为与计数，不实际改变系统进程权限。再增加一个你自己的模拟测试。

11.9 观察实验与报告模板

先按 07.9 固定环境、间隔和工作负载，做 3 次轻载与 3 次正常使用状态下的采样。使用不同输出文件名，防止后一次覆盖前一次。保留数据来源与时间，不要用参考答案填实测值。

记录项	你应填写的内容
目标	想验证哪项行为或假设
环境	操作系统、Python、依赖、是否 WSL/容器
方法	命令、参数、次数、观察窗口
预期	执行前定义成功/失败标准
事实	输出、状态、日志摘录、真实指标
解释	哪些是确认事实，哪些只是可能原因
修复与回归	修改了什么，哪些测试证明没有破坏原功能
局限	采样、可见范围、无法观察的硬件等

11.10 本周交付物

自己的采集器源码；环境与依赖说明；至少五项自己编写的测试；可解析的报告样本；五个故障实验记录；至少三次有意义的 Git 提交；一份包含已知限制的 README。参考工程和参考测试可辅助，但不能代替你的独立实现与说明。

12｜结课考核：先独立完成，再对答案

本章摘要： 结课标准是“能解释、能实现、能验证、能排错”，不是阅读进度或框架数量。

12.1 闭卷解释题 Q01~Q12

Q01：终端、Shell、Python 解释器分别负责什么？为什么在 >>> 中输入 ls 会失败？

Q02：虚拟环境隔离的是什么？如何确认当前 pip 与当前 Python 对应？

Q03：绝对路径和相对路径怎样解释？Python 的相对输出路径默认相对于哪里？

Q04：>、>>、2>、| 分别处理什么？为什么不能 cat a.txt > a.txt？

Q05：print 与 return 有什么区别？类型注解会自动验证百分比范围吗？

Q06：为什么 with 能帮助避免文件句柄泄漏？它是否保证断电时文件完整？

Q07：哪些失败允许继续，哪些必须失败退出？怎样避免“假成功”？

Q08：进程与线程有什么区别？多个线程是否一定让 Python 计算更快？

Q09：CPU 单次使用率能否证明瓶颈？内存 free 与 available 为什么不同？

Q10：RSS 持续增加能否直接证明泄漏？所有进程 RSS 相加是否等于系统已用内存？

Q11：tmp_path 与 monkeypatch 各解决什么模拟问题？否则测试可能污染真实文件正常？

Q12：git add、commit 和远程上传是否相同？ignore 能否删除已提交的密码？

回答每题用自己的话讲 30~60 秒，不要求背教材原句。能给一个项目中的具体例子，比术语堆叠更重要。

12.2 独立实操 P01~P06

P01：新建个人实验目录，建立虚拟环境，输出 Python 可执行路径和依赖版本。

P02：不用复制完整参考工程，写出采集 CPU、内存、磁盘并保存 JSON 的最小程序。

P03：加入阈值参数及范围校验；边界 80 符合 >= 规则；非法输入不能伪装成 0%。

P04：加入一次可控故障，说明退出状态、stderr、日志或日志不可用时的行为。

P05：自己写五个测试，覆盖正常、边界、异常、JSON 回读和一个可控外部失败。

P06：写 README，提交代码和脱敏实验记录，解释其中一次改动的原因与验证方法。

12.3 自评量规（教学标准，不是企业录用标准）

项目	分值	得分依据
独立实现与解释	30	能写最小程序，能解释字段与数据流
自动化测试	20	有自己设计的正常、边界、异常测试
故障诊断	20	有证据、有合理退出行为、不吞错误
系统指标理解	15	单位、采样范围、内存与进程概念正确
文档与版本	15	环境可复现，Git 改动可解释，局限明确

建议达到 80 分再进入第二周；这是学习节奏建议。即使分数够，若仍把采集失败写成正常 0 值、无法说明虚拟环境、不会保存与读取 JSON，应先补齐，不用靠后续框架掩盖基础缺口。

12.4 最常见的卡点与回看位置

症状	优先核查	回看
pip 装了但 import 失败	解释器路径、虚拟环境	01.5
文件明明有却找不到	当前目录、大小写、绝对路径	02.1、04.1
ModuleNotFoundError: monitor	是否从项目根目录启动	05.2~05.3
JSON 文件空了	是否过早用 w 打开、是否写入失败	04.2~04.5
INFO 日志不显示	日志级别、handler 与配置位置	06.4~06.5
CPU 第一次读数怪异	是否非阻塞首次调用	07.4
free 很少就认为内存故障	available、缓存与持续趋势	07.5~07.6
测试每次结果不同	是否断言了真实动态读数	09.5~09.6
command not found	工具是否安装、PATH 与所在系统	01、02.5
systemctl 不能运行	是否 systemd 环境，无需强行修复	02.7

第一周真正结束的标志：你能说明“程序在哪运行、数据怎样来、失败如何表现、结论靠什么验证”。第二周再加入网络诊断，这套基础会直接复用。

附录 A｜练习与自测参考答案

先完成再阅读。 这些答案强调判断理由；具体硬件读数不存在统一正确数字。

A.1 章节练习 E01~E10

E01 环境。 新终端先 cd ~/ai-systems-lab/week1，再 source .venv/bin/activate，用 python -c "import sys; print(sys.executable)" 检查路径，再 python -m pip --version。安装成功但导入失败，常见原因是运行环境不是安装环境，也可能是包名与导入名不同或本地同名文件遮蔽；需看证据再判断。

E02 命令组合。 下面是一套可能答案，目录限定在个人实验目录：

```
mkdir -p scratch
printf 'INFO begin\nERROR missing\nINFO finish\n' > scratch/notes.txt
grep 'ERROR' scratch/notes.txt > scratch/selected.txt
ls /week1_nonexistent_path 2> scratch/command_error.txt
```

> 覆盖正常输出，>> 追加正常输出，2> 保存错误输出。grep 找到一行 ERROR；ls 失败不代表 grep 失败。检查命令状态要紧随每条命令，不能在本组全部完成后反推前面每条结果。

E03 判断函数。 参考 lab/monitor/rules.py。先拒绝非数值/布尔值，再用 math.isfinite 排除 NaN 和无穷大，最后验证 0~100。bool 在 Python 中与整数有类型关系，本项目特意拒绝它，避免 True 被当成 1%。79.9 返回 normal，80 和 100 返回 high，非法值抛 ValueError。

E04 存储。 测试数据可用 {"note":"测试","items":[1,2],"ok":True,"gpu":None}。回读后与原字典比较。无效 JSON 通常触发 JSONDecodeError；父目录被普通文件占据通常触发 OSError 的某个子类。不同系统可能给出不同子类和消息，不把错误文本写死为通用标准。

E05 模块。 python -c "import monitor.collector" 应只完成导入，没有主动采样保存；python -m monitor 才调用入口。将函数定义放模块，将运行步骤放 main，就能实现这种区分。__init__.py 不能在导入时偷偷调用 collect。

E06 失败策略。 非法阈值使本次任务失败；整体输出失败也应失败；单个进程不可读可以继续，但报告要记录跳过数量与范围限制。日志打不开时用 stderr 给出错误，不宣称日志一定存在。实际状态依项目约定，不能只凭屏幕上有没有红字判断。

E07 指标实验。 合格结论例如："相同采样间隔下，第二组内存使用率较高；同期新增了一个程序，但其他活动未完全控制，不能据此确定全部差值都来自它。"不合格结论是"CPU 到了 80 所以 CPU 坏了"或"RSS 涨了两次所以确定泄漏"。

E08 三种机制。 生成器被消耗后不会自动复位。装饰器必须 return 原函数结果；finally 中计时不应覆盖返回值。示例上下文管理器在异常路径仍输出 end，但原 ValueError 继续向外传播；清理与抑制异常是两回事。

E09 测试。 阈值 0 时 0 应 high；阈值 100 时 99.9 应 normal、100 应 high；非法阈值应抛 ValueError。JSON 中文回读与原对象相等；父级是普通文件时 save_json 抛 OSError。把>=改成>后，恰好等于阈值的案例必须失败，否则测试没有覆盖边界。

E10 文档。 README 至少能让新环境定位目录、安装依赖、执行采集、找到报告、运行测试。补齐遗漏时应记录修改，例如“docs: clarify project root and virtualenv activation”。本地 commit 足够本周验收，不要求公开上传个人运行信息。

A.2 闭卷题 Q01~Q12 的关键点

Q01：终端提供交互，Shell 解释系统命令，Python 执行 Python 语法。>>> 所处解释器不认识普通 Shell 命令。

Q02：venv 主要隔离项目 Python 包与相关运行配置，不隔离内核。用 sys.executable 和 python -m pip 核对环境。

Q03：绝对路径从根目录出发，相对路径从当前工作目录解释；不是自动从脚本所在目录解释。

Q04：分别是覆盖 stdout、追加 stdout、重定向 stderr、连接 stdout 到 stdin。对同一文件重定向写入会先截断，再读就失去原内容。

Q05：print 展示，return 把对象交给调用者；没有 return 通常得到 None。类型注解不自动验证范围。

Q06：with 在正常和常规异常离开时调用退出清理，减少忘记 close。它不等于原子写入或断电保护。

Q07：局部进程失败可降级；核心采集或报告保存失败应非零退出。必须留下状态和证据，不能用 0 读数或空结果假装正常。

Q08：进程与线程的资源隔离和共享范围不同；并发不等于并行，多线程收益依运行时、GIL、扩展库和任务性质而定。

Q09：单次 CPU 值缺少持续趋势和业务上下文；available 还考虑可回收内存，与 free 不同。

Q10：持续增长需进一步验证，缓存和分配器行为也可能导致；RSS 含共享页，不能简单相加等同系统内存。

Q11：tmp_path 隔离文件资源，monkeypatch 控制外部依赖。模拟测试不是对真实硬件、驱动和部署环境的证明。

Q12：add 暂存，commit 保存本地历史，上传是另外的远程操作；ignore 不抹除已有历史中的秘密。

A.3 实操题的判定，不给虚构实验值

P01 以解释器路径和版本记录判定；P02 以可解析 JSON 及字段解释判定；P03 以边界与非法值测试判定；P04 以实际状态、错误输出和可追溯记录判定；P05 以自己设计的用例及

故意引入错误后能否捕获判定；P06 以新环境能否按 README 复现以及提交差异能否解释判定。

不要求你的 CPU、内存与教材机器一致；不得把教材测试环境的数值冒充你的实验结果。使用 AI 协助时，记录自己修改了什么，并能解释每条关键语句。

附录 B｜速查：最后复习时看，不替代正文

B.1 20 个本周常用操作

目的	命令或表达式
当前目录	pwd / Path.cwd()
当前解释器	python -c "import sys; print(sys.executable)"
建立/激活环境	python3 -m venv .venv / source .venv/bin/activate
查看文件含隐藏项	ls -lah
建立父目录	mkdir -p reports logs
查看开头/末尾	head -n 10 file / tail -n 10 file
筛选行	grep 'ERROR' file
检查上个命令状态	echo $?
查命令来自哪里	command -v python
查看内存	free -h
查看当前文件系统	df -h .
查看指定进程	ps -p PID -o pid,ppid,stat,comm
系统与架构	uname -a / uname -m
JSON 格式验证	python -m json.tool reports/system_report.json
运行项目	python -m monitor --help
执行测试	python -m pytest -q
看未暂存差异	git diff
看暂存差异	git diff --cached
看仓库状态	git status
看提交历史	git log --oneline

表中的 file、PID 是占位符，必须换成自己的路径或刚启动的进程号。不要直接用未知 PID 终止进程。

B.2 15 个容易混淆的概念

终端不等于 Shell；Linux 内核不等于整个 Ubuntu；venv 不等于虚拟机；字典不等于 JSON 文本；None 不等于 0；print 不等于 return；注解不等于运行时验证；with 不等于吞异常；日志不等于测试；后台进程不等于受管理服务；并发不等于并行；CPU 利用率不等于吞吐率；free 不等于 available；空间使用率不等于 IO 性能；测试通过不等于没有缺陷。

B.3 开始下一周前的最小检查

能够从新终端进入项目和激活环境；能说清每个文件职责；能新增一个指标或规则；能写对应测试；能复现并解释至少一种失败；能找到报告和日志；知道哪些结论现有数据还不能支持。满足这些，第一周的基础就已经建立起来。

附录 C｜官方资料与技术依据

正文的 [S 编号] 对应下面资料。它们用于核对语义和处理环境差异，**不是额外的必看清单**。先完成正文与实验，再按卡住的问题阅读对应小节。教程中的场景、任务、测试与判断标准为本课程编写，不是引用的岗位录用标准。

文档核对日期：2026-10-29。Python 语法讲解主要使用 3.12 版文档，代码最低语法要求为 3.10；实际执行环境另见 VALIDATION。软件更新后请核对与你安装版本对应的文档，不因在线文档版本号较高就盲目升级系统。

C.1 环境、Linux 与 Shell

[S01] Microsoft Learn：Install WSL。只需核对安装、发行版和 WSL 版本查看。

[S02] Python：Virtual Environments and Packages。对应虚拟环境、pip 和依赖记录。

[S03] Linux Kernel：The /proc Filesystem。对应系统、进程与内存信息接口。

[S04] GNU Bash：Redirections。对应 stdout/stderr、覆盖、追加和重定向顺序；管道细节同时参照 S05。

[S05] Bash 上游手册：bash(1)。对应环境变量、引用、管道、退出状态与作业控制。

[S06] GNU chmod 手册 与 Linux 路径解析手册。对应权限位、目录搜索权限及路径错误。

[S07] OpenBSD/OpenSSH：ssh(1)。对应远程登录、端口与主机验证。

[S08] Microsoft Learn：systemd in WSL。对应服务管理的环境前提。

C.2 Python 基础与工程语义

[S09] Python：More Control Flow Tools。对应分支、循环、函数参数与默认值。

[S10] Python：Data Structures。对应列表、字典、复制和遍历。

[S11] Python：Classes。对应作用域、类、实例、迭代器和生成器。

[S12] Python：typing。明确类型注解通常不由运行时强制执行。

[S13] Python：pathlib。对应路径、目录创建与文件操作。

[S14] Python：Input and Output。对应文件模式、编码、with 及 JSON 入门。

[S15] Python：contextlib。对应上下文管理协议及 contextmanager。

[S16] Python：json。对应序列化、反序列化和非有限浮点数处理。

[S17] Python：Modules。对应导入、包、__name__与脚本组织。

[S18] Python：Errors and Exceptions。对应异常类型、捕获、清理和传播。

[S19] Python：Logging HOWTO。对应级别、日志用途与配置。

[S20] Python：logging reference。对应 logger、handler、formatter 和异常记录。

C.3 资源、测试和版本管理

[S21] psutil 官方文档。对应 CPU 采样、内存 available、RSS、磁盘与进程访问异常。实践请与锁定依赖版本配套。

[S22] Python：threading。对应线程与常规 CPython/GIL 条件；不是对所有实现与构建的绝对结论。

[S23] Linux：signal(7)。对应中断、终止与信号处置。

[S24] Python：time。对应测量耗时与时钟选择。

[S25] Python：functools。对应 wraps 及包装函数元信息。

[S26] pytest：Get Started。对应测试发现、assert 和执行方式。

[S27] pytest：Parametrize。对应多组输入与独立案例。

[S28] pytest：tmp_path。对应临时目录 fixture。

[S29] pytest：monkeypatch。对应外部依赖替换与自动恢复。

[S30] Pro Git：Recording Changes。对应暂存、提交、差异和忽略规则。

[S31] Pro Git：Basic Branching and Merging。对应分支与合并。

[S32] Python：argparse。对应参数定义、解析、帮助与格式错误。

[S33] NVIDIA：CUDA Programming Guide。仅用于 CPU/GPU 异构计算背景；CUDA 编程不是第一周任务。

结束语： 先用自己的手完成一个小而可靠的程序，再逐渐扩展到 AI 系统。每次学到新技术都问四个问题：它解决什么问题？怎样工作？如何使用？出问题怎样验证原因？